

Code Generation Prompts

Create a detailed code generation prompts for the above development plan. Ensure each code generation prompt has a distinct work package number and name, such as WP-1.1 and "Phase 0: Platform setup"

Claude Opus 4.8

I'll review the development plan first, then create the code generation prompts. Let me get the rest of the plan. Here are detailed code generation prompts derived from the ClientReady Development Plan. Each work package (WP) maps to a phase and a discrete buildable slice, with a distinct number, name, and a ready-to-use prompt for a code-generation assistant.

Phase 0 — Discovery & Vendor Enablement

WP-0.1 — "Phase 0: Repository & Specification Scaffolding"

Generate a monorepo scaffold for "ClientReady" using pnpm workspaces with three packages: apps/api (Node.js + TypeScript, Express or Fastify), apps/web (React + TypeScript + Vite), and packages/shared (shared TS types). Include: root tsconfig with path aliases, ESLint + Prettier, Husky pre-commit hooks, EditorConfig, a .env.example documenting Supabase, S3, queue, and KMS variables, and a README describing the modular-monolith architecture. Also generate skeleton markdown specification files under /docs for: architecture-decision-record.md, role-permission-matrix.md, tenant-isolation-spec.md, and erp-common-data-model.md, each with structured headings ready to fill in. Do not implement business logic yet — only the structure, tooling config, and typed placeholders.

WP-0.2 — "Phase 0: Canonical Domain Type Definitions"

In packages/shared, generate TypeScript type definitions and Zod schemas for the core ClientReady domain: Tenant, User, Role, Permission, ClientAccount, Project, Job/Requisition, CandidateApplication, Interview, InterviewSession, Recording, Review, Decision, Document, AuditEvent, and ERP connector entities (Requisition, Candidate, Employee, InterviewRequest, ReadinessStatus, SyncState, ExternalReference, ConnectorInstallation). Every entity must carry tenantId, and where relevant clientId and projectId, as required scope fields. Export both the TS interfaces and matching runtime Zod

validators. Add enums for candidate status workflow, review status, and decision outcomes. Include JSDoc comments explaining each field.

Phase 1 — Platform Foundation

WP-1.1 — "Phase 1: Database Schema & Row-Level Security"

Generate a set of PostgreSQL/Supabase SQL migration files for ClientReady's foundational schema: tenants, users, roles, permissions, role_assignments, client_accounts, projects, project_memberships, and audit_events tables. Every tenant-scoped table must include tenant_id (and client_id/project_id where applicable) with foreign keys and indexes. Enable Row-Level Security on all tables and write RLS policies that restrict access by the current tenant/client/project context set via a Postgres session variable (e.g. app.current_tenant). Include the pgvector extension enablement and a documents_embeddings table with a vector column. Provide up and down migrations, and a seed script for a demo tenant, roles, and an admin user.

WP-1.2 — "Phase 1: Tenant-Aware Request Context & Auth Middleware"

In apps/api, generate Node.js + TypeScript middleware that establishes a tenant-aware request context for every request. It must: verify the authenticated session (Supabase Auth JWT), derive tenantId, clientId, and projectId strictly from the user's server-side role assignments (never from client-supplied identifiers), attach an immutable RequestContext object, and set the Postgres session variable used by RLS. Include an authorization guard factory that checks role-based and resource-based permissions against the permission matrix, plus a strict rule that rejects any request attempting to pass scope identifiers in the body/query. Add unit tests proving scope is server-derived and cannot be overridden by the client.

WP-1.3 — "Phase 1: Identity, Tenant Onboarding & Client/Project Management APIs"

Generate the API modules for identity and tenant management in the modular monolith: endpoints for tenant onboarding, user invitation/creation, role assignment, client-account creation, and project creation/management (including whether an employee can belong to multiple concurrent projects — make this configurable). Use a layered structure (router → controller → service → repository) per module. Enforce authorization via the guards from WP-1.2, emit audit events for every sensitive action, and add centralized request validation (Zod) and a centralized error handler with typed error responses. Include API versioning (/v1) and idempotency-key support for POST endpoints.

WP-1.4 — "Phase 1: Audit Event Generation & Query Service"

Generate an audit subsystem in apps/api: an AuditService that writes structured, append-only audit events (actor, tenant/client/project scope, action, resource, outcome, timestamp, request id) for workflow transitions, access grants, and admin actions. Provide a decorator/helper to instrument service methods, a write path that never blocks the main transaction on failure, and a scoped query API for admins to view audit history. Ensure no sensitive payload content is logged by default. Include tests.

WP-1.5 — "Phase 1: React Application Shell, Auth & Role-Aware Navigation"

Generate a React + TypeScript (Vite) application shell for ClientReady with: Supabase auth integration (login, logout, session refresh), a routing layer, a role-aware navigation component that renders menu items based on the user's permissions, protected route wrappers, an accessible responsive layout foundation, and a reusable design-system component library (buttons, forms, tables, modals, toasts) with a theme. Include tenant-administration and client/project-administration screens wired to the WP-1.3 APIs. Add loading/error states and a11y baseline (focus management, ARIA, keyboard nav).

WP-1.6 — "Phase 1: CI/CD, IaC & Observability Baseline"

Generate DevOps assets for ClientReady: GitHub Actions pipelines for lint, test, build, dependency + container scanning, and deploy to dev/staging/production; Infrastructure-as-Code (Terraform) defining the Supabase/Postgres, S3-compatible bucket, managed queue, and KMS resources with isolated environments; a database migration runner step; secrets management wiring; and an observability bootstrap (structured logging, OpenTelemetry tracing, metrics) shared across the API. Include a backup-and-recovery baseline job and documentation.

Phase 2 — Resourcing Workflow MVP

WP-2.1 — "Phase 2: Job & Requisition Workflow"

Generate the job/requisition module: APIs and services for job-code creation, publication, editing, and lifecycle status. Include validation, scope enforcement, audit events, and pagination/filtering. Add React screens for creating, publishing, and listing jobs with role-aware actions. Include tests covering publication and permission boundaries.

WP-2.2 — "Phase 2: Candidate Application, Triage & Shortlisting"

Generate the candidate application and triage module: endpoints to submit applications against a published job, triage applicants (accept/reject/hold), and manage a shortlist. Implement the candidate status workflow state machine with allowed transitions and audit logging. Build React views for the applicant pipeline (Kanban-style triage board), shortlist management, and candidate detail. Include unit tests for the state machine.

WP-2.3 — "Phase 2: Interview Assignment, Scheduling & Question Bank"

Generate the interview preparation module: interview assignment to candidates, preparation-window scheduling, panelist assignment, and question-bank administration (CRUD for reusable questions and per-job question sets, supporting a 10-question interview configuration). Provide APIs, services with scope enforcement, and React admin UIs. Include notifications/reminders hooks (interface + stub implementation) and tests.

WP-2.4 — "Phase 2: Structured Grading & Final Selection"

Generate the review/grading/decisioning module: configurable scoring rubric per job, reviewer scoring capture, aggregated scores, final candidate selection, and a recorded decision with rationale. Enforce that only authorized reviewers can grade, emit audit events for scores and decisions, and expose read APIs for decision history. Add React grading and decision screens. Include tests.

WP-2.5 — "Phase 2: External Agency Portal & Temporary Candidate Access"

Generate the external agency capability: agency organization and user onboarding, scoped access limited to specifically shared resource requests, CV upload, candidate submission, temporary candidate credentials with automatic expiration, and full audit history. Implement time-boxed access tokens that auto-revoke, strict scope isolation (an agency sees only assigned requests), and a dedicated React agency portal. Include negative tests proving no access outside granted scope and correct credential expiry.

WP-2.6 — "Phase 2: Tenant Usage Reporting"

Generate a basic tenant-level usage reporting module: capture usage/billing metrics (jobs, applications, interviews, storage), aggregate them per tenant, and expose reporting APIs plus a React dashboard with charts. Ensure all queries are tenant-scoped and audited. Include tests.

Phase 3 — Interview Recording & Review

WP-3.1 — "Phase 3: Candidate Interview Recording Experience"

Generate the candidate interview client in React: pre-interview device checks, camera/microphone permission flow, a sequential 10-question interview session, per-question recording controls (record/re-record within policy), upload progress with retry, session recovery after interruption, candidate consent capture, review of permitted recordings, and a submission confirmation screen. Use MediaRecorder, handle browser inconsistencies gracefully, and support resumable uploads. Include a supported-browser policy check.

WP-3.2 — "Phase 3: Direct-to-Storage Media Upload & Processing Pipeline"

Generate the media architecture backend: an API that issues short-lived signed upload instructions so browsers upload recordings directly to S3-compatible storage; background workers (Node.js + managed queue) for media validation, transcoding/normalization, and metadata extraction; and asynchronous processing that never blocks the API. Store retention metadata per recording. Include idempotent worker handlers, dead-letter handling, and tests with mocked storage.

WP-3.3 — "Phase 3: Secure Playback & Reviewer Experience"

Generate secure video playback and the reviewer UI: an API that returns short-lived authorized playback links only to users with a verified review assignment, and a React reviewer experience with question-by-question navigation, a configurable scoring rubric, reviewer comments, overall recommendation, review status, and the final decision workflow. Log view events for audit. Include negative tests proving unauthorized users cannot obtain playback links or scores.

WP-3.4 — "Phase 3: Retention, Archive & Restore Lifecycle Automation"

Generate lifecycle automation for recordings based on these rules (make each configurable and confirm the calculation start point via config): default retention up to 10 days; expected review 3–4 days; selected candidate +7 days; other recordings deleted after the applicable window; archived deleted recordings kept ~1 month; restore within 30–60 minutes. Implement scheduled workers that apply retention, move to archive, and process restore-request-and-approval workflows, with full audit trails. Include tests simulating each lifecycle transition with a mocked clock.

Phase 4 — AI/RAG & Domain Learning

WP-4.1 — "Phase 4: Scoped Document Ingestion Pipeline"

Generate the document ingestion pipeline: firm-, client-, and project-level upload with role-specific permissions, malware scanning, file-type/size controls, text extraction, chunking, embedding via a provider-neutral interface, scope-metadata assignment, encryption before persistence, ingestion-status tracking, and document versioning/deletion. Store chunks + embeddings in pgvector with scope metadata columns. Use background workers for extraction/embedding. Include tests.

WP-4.2 — "Phase 4: Provider-Neutral AI Abstraction Layer"

Generate a provider-neutral AI abstraction in packages/shared or a dedicated module: interfaces for EmbeddingProvider and LLMProvider with pluggable implementations, configuration-driven provider selection, retry/timeout handling, token/cost accounting hooks, and structured logging that avoids logging sensitive prompt/document content by default. Include a mock provider for tests.

WP-4.3 — "Phase 4: Server-Enforced Scoped RAG Retrieval"

Generate the scoped RAG retrieval service implementing this exact server-side flow: (1) validate the user's interview/learning assignment, (2) resolve firm/client/project context from the session, (3) determine allowed inherited scopes, (4) build an enforced server-side retrieval filter, (5) retrieve only in-scope chunks from pgvector, (6) decrypt only authorized chunks, (7) send minimum necessary context to the LLM, (8) return a grounded answer with source references, (9) log scope/documents/outcome, (10) never log sensitive content by default. Scope must never come from the browser. Include tests.

WP-4.4 — "Phase 4: Domain-Learning Assistant & Practice Sessions"

Generate the learning capabilities on top of scoped RAG: a scoped domain-learning assistant, practice-question generation, suggested answer criteria, source references, employee practice sessions, grounded preparation content, and content feedback/quality reporting. Provide APIs and React UIs. Ensure every generation is grounded in in-scope documents only. Include tests.

WP-4.5 — "Phase 4: RAG Cross-Scope Negative Test Suite"

Generate an automated negative test suite (blocking release) covering: Tenant A retrieving Tenant B content; Client A vs Client B; Project A vs Project B; higher-level user entering a lower-level session; prompt-injection attempts requesting hidden material; manipulated client-side scope identifiers; deleted/revoked document access; stale cached retrieval results; and search-index metadata tampering. Each test must assert zero cross-scope leakage and that retrieval logs prove enforced scope. Wire it into CI as a required gate.

Phase 5 — ERP-Neutral Connector Platform

WP-5.1 — "Phase 5: Canonical Integration Contract & Connector SDK"

Generate an internal TypeScript package for the ERP-neutral connector platform containing: the canonical/vendor-neutral model types (Job Requisition, Candidate, Employee, Interview Request, Readiness Status, Sync State, External Reference, Connector Installation, Connector Authorization), an Adapter interface every ERP must implement, authentication helpers, a retry strategy, idempotency handling, structured logging, observability hooks, a test harness, contract tests, and mock ERP responses. Include a connector certification checklist as documentation. Keep fields extensible pending per-ERP API review.

WP-5.2 — "Phase 5: Synchronization Engine (Webhooks, Polling, Retry, Reconciliation)"

Generate the connector synchronization engine: a webhook/event receiver, scheduled polling workers for ERPs without webhooks, per-tenant OAuth/credential management with encrypted storage, idempotency controls, retry with dead-letter processing, per-tenant rate limits, and reconciliation logic. Ensure replayed events never duplicate records and expired credentials are handled safely. Include tests using the mock ERP from WP-5.1.

WP-5.3 — "Phase 5: Connector Administration Console & Health Dashboard"

Generate tenant-admin connector management (API + React): install/connect workflow, connection status, last successful synchronization, scope/permission summary, error and remediation messages, manual resynchronization, disconnect with credential revocation, mapping review, connector health/reconciliation dashboard, and data-processing disclosure. Enforce that no connector can resolve another tenant's credentials or data. Include tests for isolation and revocation.

Phase 6 — First Marketplace Connector

WP-6.1 — "Phase 6: First Vendor Adapter Implementation"

Generate a concrete ERP adapter implementing the WP-5.1 interface for the first target ERP (parameterize the vendor; default to a mock/sandbox until real sandbox access is confirmed). Implement installation/authorization, OAuth token acquisition and refresh, requisition retrieval, candidate/employee reference retrieval, identity mapping, status synchronization, webhook or polling as supported, rate-limit handling, retry/dead-letter, reconciliation, health reporting, and credential revocation. Run it against the shared contract and security test suites.

WP-6.2 — "Phase 6: Thin Marketplace Extension & Deep Link"

Generate the intentionally thin marketplace extension: tenant connection and authorization, limited requisition import, limited candidate/employee reference import, interview-request initiation, readiness-status display, a deep link into the hosted ClientReady experience, and installation/support information. Explicitly exclude video, RAG, and complex grading from the extension. Include uninstall and credential-revocation behavior. Provide the data-flow diagram and privacy/security documentation stubs required for marketplace submission.

Phase 7 — Second & Third ERP Connectors

WP-7.1 — "Phase 7: Additional Vendor Adapter (Reusing Shared Platform)"

Generate the next vendor-specific ERP adapter reusing the common integration contract, connector SDK, sync engine, and health/reconciliation features from Phases 5–6. Implement ONLY vendor-specific authentication and mapping. Run the identical contract and security test suites and produce the marketplace submission artifacts. Make the target ERP a parameter so this template can be reused for the Wave 1 (Workday, SAP SuccessFactors, Oracle HCM) and Wave 2 (ADP Workforce Now, UKG Pro, Dayforce) sequence.

Phase 8 — Client Stakeholder Experience & Pilot Readiness

WP-8.1 — "Phase 8: Restricted Client-Stakeholder Video Viewer"

Generate the restricted client-stakeholder experience: invitation tied to exactly one client request, explicit per-candidate-video grant, expiring access, short-lived playback authorization, optional watermarking, a view-event audit trail, grant revocation, and a controlled download policy. Enforce NO browsing across jobs/candidates/projects and require terms/consent acknowledgement. Provide API + a minimal React viewer. Include negative tests proving the viewer cannot enumerate or access anything beyond the single granted video.

WP-8.2 — "Phase 8: Pilot Enablement — Runbooks, Onboarding & Monitoring"

Generate pilot enablement assets: a tenant onboarding runbook, connector installation runbook, admin/reviewer training docs, candidate instructions, a data-import validation script, a security-acceptance test checklist, a performance/load test harness (model ~4,000 recorded minutes per client/day scaling toward 20,000 interviews/day across 100 clients), retention-job validation tests, an incident-response exercise script, monitoring/alerting configuration, and pilot success-metric dashboards. Deliver as documentation plus runnable scripts where applicable.

Cross-Cutting

WP-9.1 — "Cross-Cutting: Quality & Security Gate Automation"

Generate an automated gate-enforcement suite mapped to the plan's five gates: Gate 1 (no cross-tenant API/DB/object access; role matrix approved), Gate 2 (end-to-end job-to-decision test; unauthorized users blocked from recordings/scores; temp credentials expire; complete audit trail), Gate 3 (cross-firm/client/project + prompt-injection tests; retrieval logs prove enforced scope), Gate 4 (idempotency, token refresh/revocation, rate-limit, reconciliation, tenant credential isolation), and Gate 5 (load tests, backup restoration, video lifecycle, pen-test issue closure, monitoring/alerts, incident runbook exercised). Wire each gate as a required CI stage that blocks promotion between environments.

Want me to export these as a formatted document (DOCX or PDF), or expand any single work package into a fuller spec with acceptance criteria and file-tree output?

Create a goal and acceptance criteria for the AI agent to ensure that

the code being generated is fit for purpose.

Claude Opus 4.8

Here is a master **Goal** and **Acceptance Criteria** the AI agent must satisfy for every code-generation task in ClientReady, so output is verifiably fit for purpose.

Agent Goal

Generate production-grade, tenant-isolated, security-first code for ClientReady that correctly implements the specified work package, enforces scope on the server, is tested against cross-scope leakage, and is safe to promote through the CI quality gates without human rework.

The agent is not "done" when code compiles — it is done when the code is scoped, tested, auditable, and gate-passing.

Acceptance Criteria

1. Correctness & Scope Fidelity

- Implements exactly the work package requested — no missing requirements, no unrequested scope creep.
- Every tenant-scoped entity carries tenantId (and clientId/projectId where applicable); no scoped operation runs without a resolved context.
- Scope is **derived server-side** from the authenticated session/role assignments — client-supplied scope identifiers are rejected, never trusted.

2. Security & Isolation

- Row-Level Security (or equivalent enforced filter) applies to all scoped tables; RAG retrieval and media playback are server-enforced and scope-filtered.
- Secrets, credentials, and documents are encrypted at rest; signed URLs and access tokens are short-lived and auto-expiring.

- No sensitive payload, prompt, or document content is logged by default.

3. Testing (Blocking)

- Unit and integration tests cover happy path, permission boundaries, and error handling.
- **Automated negative tests for cross-tenant / cross-client / cross-project leakage exist and pass** — this is a hard release blocker.
- Tests are deterministic (mocked clock, storage, ERP, and AI providers where relevant).

4. Auditability & Observability

- Sensitive actions (access grants, workflow transitions, decisions, admin changes) emit structured audit events with actor, scope, action, resource, outcome, and request id.
- Structured logging, tracing, and error handling are wired; audit writes never block the primary transaction.

5. Architecture & Quality

- Follows the modular-monolith layering (router → controller → service → repository) and the connector-first, provider-neutral abstractions (LLM/embedding/ERP adapters).
- Passes lint, type-check, build, and dependency/container scans; validated inputs (Zod) and typed error responses throughout.
- Public interfaces are documented; migrations include up/down; API endpoints are versioned and idempotent where mutating.

6. Gate Alignment

- Output demonstrably satisfies the relevant CI gate(s) for its phase (Gate 1 isolation → Gate 5 pilot readiness) and does not regress any prior gate.
- Retention, lifecycle, connector reconciliation, and credential revocation behaviors (where in scope) are implemented and test-verified.

7. Deliverable Integrity

- Returns runnable, complete files (no placeholders in shipped logic), with a clear file tree, setup notes, and any required env variables documented.
- Flags any assumption, external dependency (e.g., unconfirmed ERP sandbox), or unresolved requirement rather than silently guessing.

Definition of Done: the work package's functional requirements are met **and** criteria 1–7 hold, with cross-scope negative tests green in CI.

Want me to attach a tailored, single-line "Definition of Done" to each individual WP (WP-0.1 ... WP-9.1), or keep this as one shared standard?
